

SMART TRAVEL PLANNER

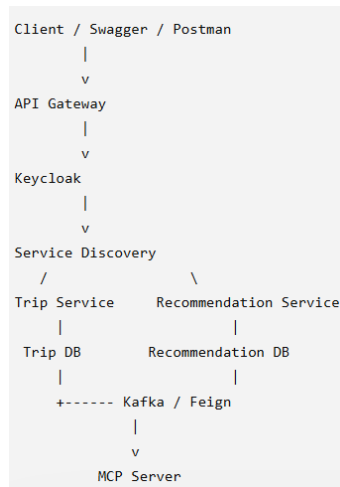
(COA проект)

1. Scope

Два сервиси:

- Trip Service
 - креирање патување;
 - внесување destination, startDate, endDate, budget;
 - преглед на патувања;
 - преглед на едно патување;
 - бришење/ажурирање патување.
- Recommendation Service
 - препораки за атракции;
 - препораки за ресторани;
 - препораки за хотели;
 - зачувување препорака за конкретно патување;
 - преглед на зачувани атракции/места;
 - проценка на трошок од препораките.

2. Архитектура



Системот е дизајниран како микросервисна архитектура составена од два главни доменски сервиси: Trip Service и Recommendation Service. Секој сервис има сопствена база на податоци и независна одговорност. Клиентот комуницира со системот преку API Gateway, кој ги рутира барањата кон соодветниот сервис. Service Discovery компонентата овозможува динамичко откривање на сервисите. За безбедност се користи Keycloak со JWT токени. Trip Service и Recommendation Service комуницираат синхроно преку REST/Feign, а за event-driven комуникација се користи Kafka.

Компоненти:

- API Gateway
- Service Discovery Server
- Trip Service
- Recommendation Service
- Trip Database
- Recommendation Database
- Kafka Broker
- Keycloak
- MCP Server

3. GitHub монорепо

```
smart-travel-planner/
├── api-gateway/
├── discovery-server/
├── trip-service/
├── recommendation-service/
├── mcp-server/
├── docker-compose.yml
├── README.md
├── docs/
│   ├── architecture-diagram.png
│   ├── specification.md
│   └── api-contracts.md
└── postman/
    └── smart-travel-planner.postman_collection.json
```

4. Endpoints

- Trip Service
 - POST /api/trips
 - GET /api/trips
 - GET /api/trips/{id}
 - PUT /api/trips/{id}
 - DELETE /api/trips/{id}
 - GET /api/trips/{id}/recommendations
 - GET /api/trips/{id}/estimated-cost
- Recommendation Service
 - GET /api/recommendations?destination=Paris&type=ATTRACTION
 - GET /api/recommendations/hotels?destination=Paris&budget=1000
 - GET /api/recommendations/restaurants?destination=Paris
 - GET /api/recommendations/attractions?destination=Paris
 - POST /api/recommendations/{id}/save
Request body: {"tripId": 1}
 - GET /api/recommendations/saved?tripId=1
 - GET /api/recommendations/estimate?tripId=1

5. Бази

- Trip Service -> Trip DB
- Recommendation Service -> Recommendation DB

Trip Service не смее директно да чита recommendation табели. Ако му требаат препораки, го повикува Recommendation Service.

6. Модели

- Trip Service (Trip)
 - id
 - userId
 - destination
 - startDate
 - endDate
 - budget
 - currency
 - status
 - createdAt
- Recommendation Service
 - Recommendation:
 - id
 - destination
 - name
 - type
 - description
 - estimatedPrice
 - rating
 - source

SavedRecommendation:

- id
- tripId
- userId
- recommendationId
- savedAt

7. Комуникација меѓу сервиси

Основна комуникација: Feign / REST

Trip Service го повикува Recommendation Service кога сака препораки.

Пример:

Trip Service -> Recommendation Service

GET /api/recommendations?destination=Paris&budget=1000

Напредна комуникација: Kafka

Кога ќе се креира trip, Trip Service праќа event:

TripCreatedEvent

- tripId
- destination
- startDate
- endDate
- budget

Recommendation Service го слуша event-от и може да подготви препораки.

8. DDD

Trip Service го претставува Trip Planning bounded context, бидејќи е одговорен за креирање и управување со патувања.

Recommendation Service го претставува Recommendation bounded context, бидејќи е одговорен за генерирање, филтрирање и зачувување препораки за хотели, ресторани и атракции.

9. MCP сервер

MCP tools:

- recommend_places(destination, type, limit)
- get_saved_attractions(tripId)
- estimate_trip_cost(tripId)
- get_trip_details(tripId)

Пример:

Корисник: „Предложи ми 3 места во Париз“

MCP server повикува:

Recommendation Service

GET /api/recommendations/attractions?destination=Paris&limit=3

Корисник: „Колку ќе ме чини патувањето?“

MCP server повикува:

Trip Service

GET /api/trips/{id}

Recommendation Service

GET /api/recommendations/estimate?tripId={id}

10. Безбедност

За автентикација и авторизација се користи Keycloak. Корисникот добива JWT token по најава. API Gateway го валидира токенот и ги пропушта барањата кон

сервисите. Сервисите го користат `userId` од токенот за да ги ограничат податоците на конкретен корисник.

11. Service Discovery

Сервисите се регистрираат во `Discovery Server`. `API Gateway` ги користи регистрираните имиња на сервисите за да ги рутира барањата. Ова овозможува сервисите да не зависат од фиксни `URL` адреси.

12. API Gateway

`API Gateway` е единствен влез во системот. Тој ги рутира барањата:

- `/api/trips/**` кон `Trip Service`
- `/api/recommendations/**` кон `Recommendation Service`
- `/mcp/**` кон `MCP Server`, доколку е потребно

13. Тестирање

За `unit testing` ќе се користи `JUnit` и `Mockito`. За `integration testing` ќе се користат `Spring Boot tests`. За тестирање на договорот меѓу `Trip Service` и `Recommendation Service` ќе се користи `Pact`. `Trip Service` е `consumer`, бидејќи бара препораки, а `Recommendation Service` е `provider`.

14. External API

Во почетната верзија `Recommendation Service` ќе користи `seed` податоци зачувани во локална база, со цел системот да биде стабилен и независен од надворешни `API keys`. Како можност за надградба, сервисот може да се поврзе со надворешно `Travel/Places API` преку посебен `adapter layer`. Овој `adapter` ќе служи како `Anti-Corruption Layer`, кој ги претвора надворешните податоци во внатрешниот `Recommendation` модел.

15. Архитектурен дијаграм

Архитектурниот дијаграм ќе ги прикаже следните компоненти:

- Client / Postman / Swagger
- API Gateway
- Keycloak
- Discovery Server
- Trip Service
- Recommendation Service
- Trip Database
- Recommendation Database
- Kafka Broker
- MCP Server
- можен External Travel API